



EMERITUS

Learn. From the world's best.

Model Evaluation



EMERITUS

Learn. From the world's best.

Introduction to Model Evaluation

Key Ideas to Understand Model Performance

Purpose

Learn how to evaluate classification model performance

Generalisation

Check if the model works well on new, unseen data

Multiple Perspectives

Use different metrics to view performance from various angles

Context Matters

Choose metrics based on the problem (e.g., class imbalance)

Confusion Matrix

Basic Tool to Evaluate Classification Results

Purpose

- Foundation for most classification metrics.

A 2x2 matrix for binary classification:

- **True Positive (TP):** Correctly predicted positive class
- **True Negative (TN):** Correctly predicted negative class
- **False Positive (FP):** Incorrectly predicted positive class
- **False Negative (FN):** Incorrectly predicted negative class

```
from sklearn.metrics import confusion_matrix  
confusion_matrix(y_true, y_pred)
```

Visual Example:

	Predicted Pos	Predicted Neg
Actual Pos	TP	FN
Actual Neg	FP	TN

Accuracy

Measures overall Prediction Correctness

- **Purpose**
Overall correctness
- **Formula**
$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$
- **Range**
Between 0 and 1 (1 means perfect accuracy)
- **Best Use**
Works well with balanced class distributions
- **Caution**
Can be misleading with imbalanced data

Implementation

```
from sklearn.metrics  
import accuracy_score  
accuracy_score(y_true,  
y_pred)
```

Precision

Accuracy of Predicted Positives



Purpose

Measures how many predicted positives are actually correct



Formula

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Range: **0 to 1**



Use Case

Useful when false positives are costly (e.g., spam filters)



Implementation

```
from sklearn.metrics import precision_score  
precision_score(y_true, y_pred)
```

Recall (Sensitivity/TPR)

How Well Actual Positives are Detected

- **Purpose**

How many actual positives are correctly identified.

- **Formula**

$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

Range: **0 to 1**

- **Critical Application**

Important when False Negatives are costly (e.g., cancer detection).

- **Code Implementation**

```
from sklearn.metrics import recall_score  
recall_score(y_true, y_pred)
```

F1 Score

Balance Between Precision and Recall

```
from sklearn.metrics import  
f1_score  
f1_score(y_true, y_pred)
```

1

Purpose

Combines Precision and Recall into one score

2

Formula

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

3

Range

0 to 1

4

Application

Harmonic mean – ideal for imbalanced data

Specificity (TNR)

Measures How well Negatives are Identified

- **Purpose**
Measures correct classification of negatives
- **Formula**
$$\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$$

Range: 0 to 1
- **Relationship**
Complements Recall

Manual calculation Example

```
def specificity_score(y_true, y_pred):  
    cm = confusion_matrix(y_true, y_pred)  
    tn, fp, fn, tp = cm.ravel()  
    return tn / (tn + fp)
```

ROC Curve - Introduction

Shows the Trade-Off Between True and False Positives

Purpose

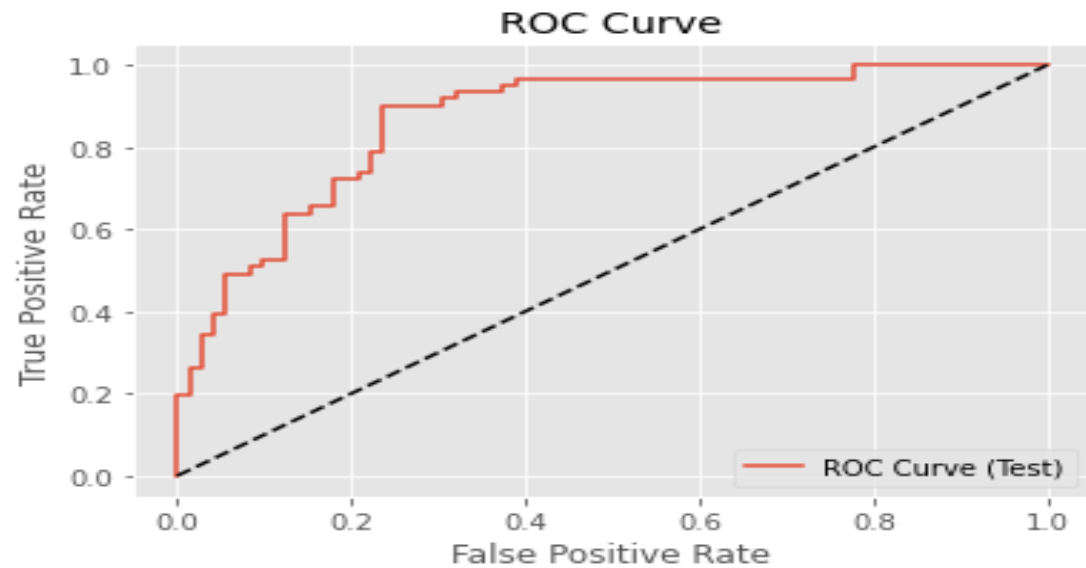
Visualise trade-off between TPR and FPR

- TPR = Recall
- $FPR = FP / (FP + TN)$
- Plot: TPR vs FPR at different thresholds.
- Range (for both TPR and FPR): **0 to 1**

Implementation

```
from sklearn.metrics import roc_curve  
fpr, tpr, thresholds = roc_curve(y_true, y_scores)
```

ROC Curve - Plotting



```
import matplotlib.pyplot as plt
plt.plot(fpr, tpr, label='ROC Curve')
plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

AUC - Area Under ROC Curve

Measures Overall Model Performance

0.5

Random Performance

Baseline random classifier

1.0

Perfect Classifier

Ideal model performance

Purpose: Single value summary of ROC curve

Range from **0.5 (random)** to **1 (perfect classifier)**

Higher AUC = better performance

```
from sklearn.metrics import roc_auc_score  
roc_auc_score(y_true, y_scores)
```

Precision-Recall Curve

Best suited for Imbalanced Datasets

Purpose

Better for
imbalanced
datasets

Trade-off Visualisation

Shows trade-
off between
precision and
recall

Range

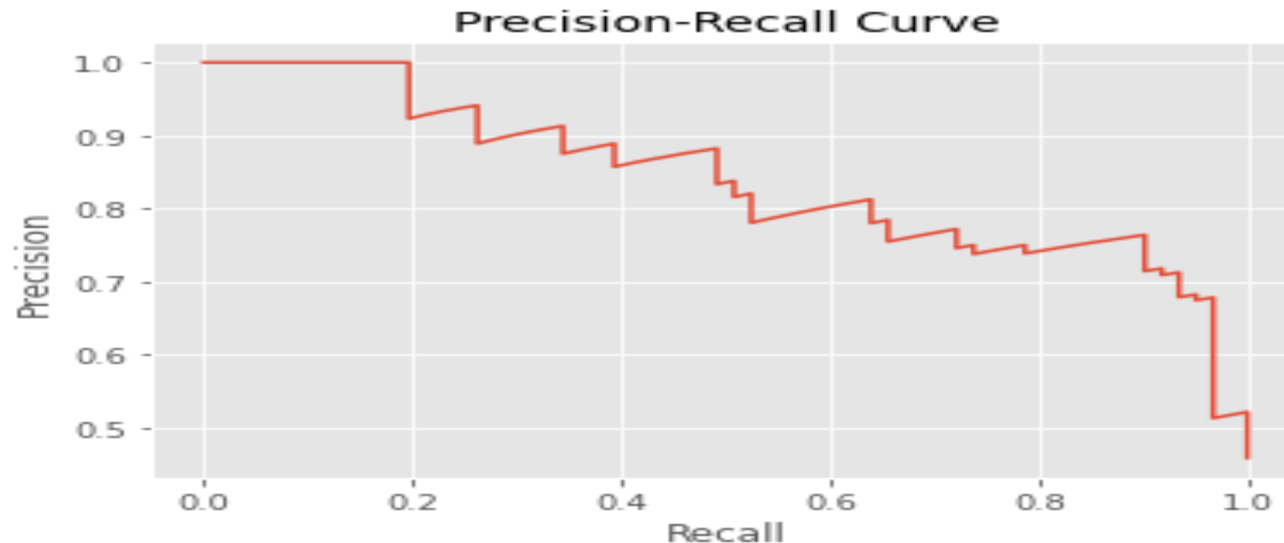
Precision and
Recall Range:
0 to 1

Precision-Recall Curve

Best suited for Imbalanced Datasets

```
from sklearn.metrics import precision_recall_curve
precision, recall, thresholds = precision_recall_curve(y_true, y_scores)

plt.plot(recall, precision)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.show()
```



Log Loss (Cross-Entropy Loss)

Measures Accuracy of Predicted Probabilities

```
from sklearn.metrics import log_loss  
log_loss(y_true, y_prob)
```

Useful for probabilistic models

- **Purpose**

Penalises false confident predictions

- **Performance Indicator**

Lower value = better model

- **Range**

0 to ∞ (0 = perfect prediction)

Matthews Correlation Coefficient (MCC)

Balanced Accuracy for Binary Classification

- **Purpose**

Balanced measure for binary classification

- **Perfect Prediction**

+1: Perfect prediction

- **Random Prediction**

0: Random prediction

- **Total Disagreement**

-1: Total disagreement

Range: **-1 to +1**

```
from sklearn.metrics import log_loss  
log_loss(y_true, y_prob)
```


Cohen's Kappa Score

Measures Agreement Beyond Chance

Purpose

- Agreement between predicted and true labels, adjusting for chance

Range: **-1 to +1**

- **+1**: Perfect agreement
- **0**: Agreement by chance
- **-1**: Complete disagreement

```
from sklearn.metrics import cohen_kappa_score  
cohen_kappa_score(y_true, y_pred)
```

Balanced Accuracy

Average Recall Across all Classes

- **Purpose**
Calculates mean recall per class
- **Application**
Imbalanced datasets

Range: **0 to 1**

```
from sklearn.metrics import balanced_accuracy_score  
balanced_accuracy_score(y_true, y_pred)
```

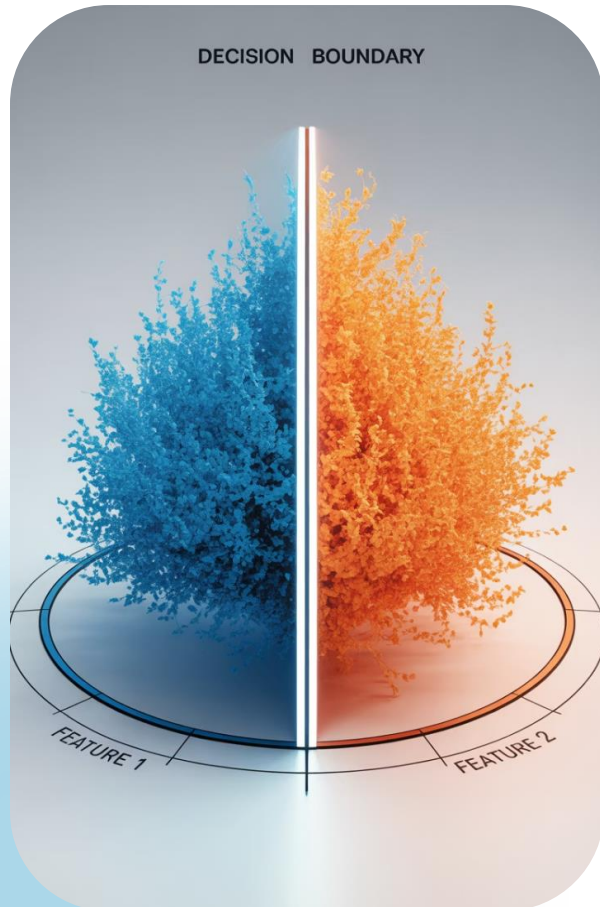
Summary of Metrics

Metric	Use Case	Range
Accuracy	Balanced classes	0 to 1
Precision	Costly false positives	0 to 1
Recall	Costly false negatives	0 to 1
F1-Score	Trade-off between precision and recall	0 to 1
ROC-AUC	Probability scores and binary classification	0.5 to 1
PR-AUC	Imbalanced datasets	0 to 1
Log Loss	Probabilistic models	0 to ∞
MCC	Balanced view even with imbalanced classes	-1 to +1
Cohen's Kappa	Inter-rater agreement	-1 to +1
Balanced Accuracy	Imbalanced data, multi-class	0 to 1

Understanding Decision Boundaries in Classification

Introduction to Decision Boundaries

Visualising How Classifiers Split Classes



Purpose

Shows how models separate classes

Key Characteristics

A surface dividing predicted classes

Understanding Models

Reveals how models generalise

Variability

Changes with model type and feature transformation

What is a Decision Boundary?

Where a Classifier Changes its Prediction

Definition

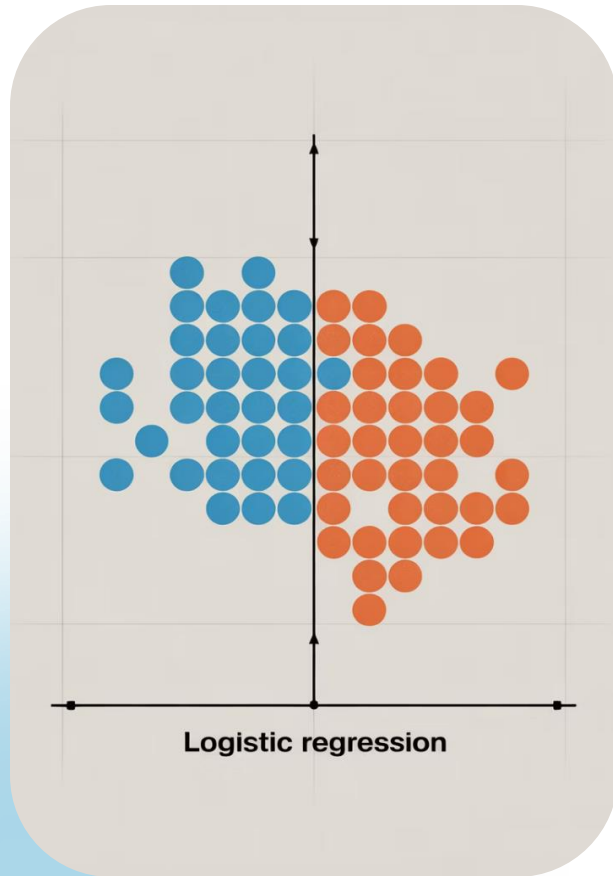
- Point where the model's class prediction shifts
- In 2D, it is typically a line (linear models) or a curve (non-linear models)

Example

- Logistic Regression: Linear boundary
- K-Nearest Neighbors: Irregular boundary based on local data structure

Linear Decision Boundaries

Straight-Line Separation for Linearly Separable Data



Model

- Logistic Regression or Linear SVM

Characteristics

- Uses a straight line or hyperplane
- Works well when data is linearly separable

Implementation

```
from sklearn.linear_model import LogisticRegression
```

Visual: Line dividing two clusters

Non-Linear Decision Boundaries

Handling Complex Class Separation



- **Models Types**

Kernel SVM, Decision Trees, KNN



- **Capability**

Captures curved, complex boundaries



- **Application**

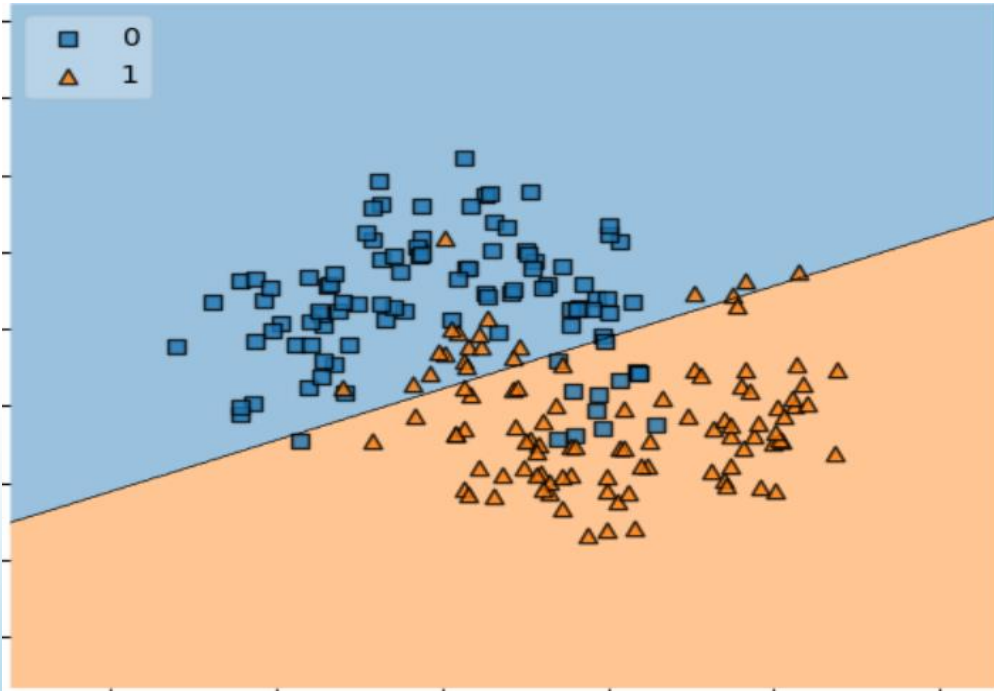
Works well with non-linearly separable data

```
from sklearn.svm import SVC # with kernel='rbf'
```

Visual: Curved boundary fitting data contours

Decision Boundary Visualisation

Understanding Classifier Predictions



- **Tools**

mlxtend, matplotlib, numpy

- **Implementation**

```
from mlxtend.plotting import plot_decision_regions  
plot_decision_regions(X, y, clf=model, legend=2)
```

- **Interpretation**

Colours show predicted class areas

- **Purpose**

Reveals how model separates features

Effect of Model Complexity

How Simplicity vs. Complexity Shapes Boundaries

Simple Model

Smooth, linear boundaries

Complex Model

Tight contours, risk of overfitting

Visual: Logistic Regression vs. High-Degree Polynomial SVM

Overfitting and Decision Boundaries

When the Model Learns Too Much

Poor Generalisation: Struggles on unseen data

Perfect Fit: Captures noise in training data

Complex Boundaries: Jagged and overly detailed

Visual: Train vs. Test – erratic test behaviour

Decision Boundaries by Classifier

How Different Models Split the Feature Space

Classifier	Decision Boundary
Logistic Regression	Linear
KNN (k=1)	Highly irregular
KNN (k=15)	Smoother
Decision Tree	Axis-aligned rectangles
SVM (RBF Kernel)	Smooth, curved

Feature Transformation and Boundary Shape

Making Models More Flexible

Transformation: Methods Polynomial features or kernel tricks

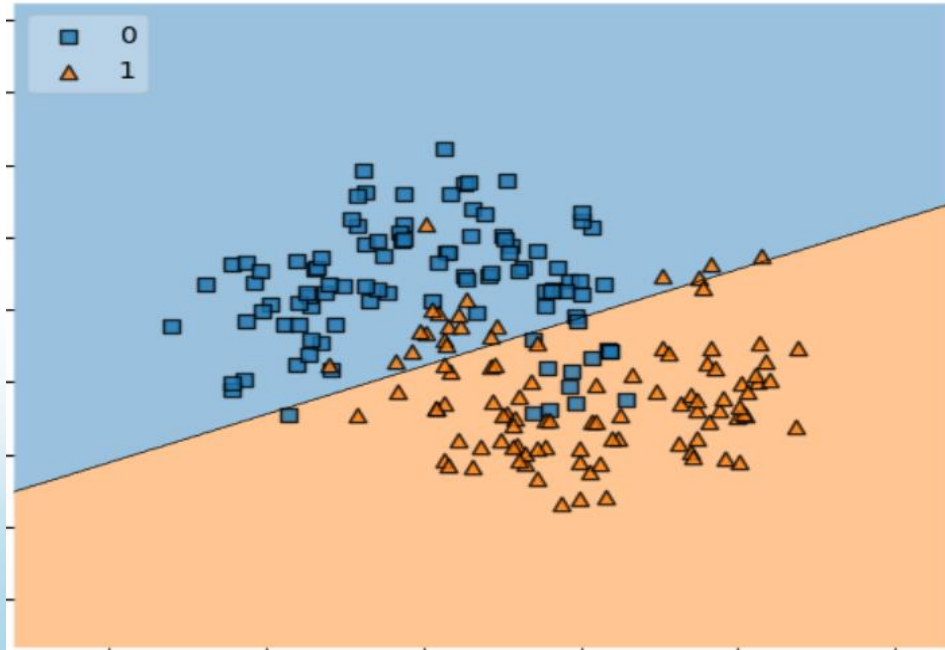
Enhanced Expressiveness: Creates more expressive boundaries

Implementation Example

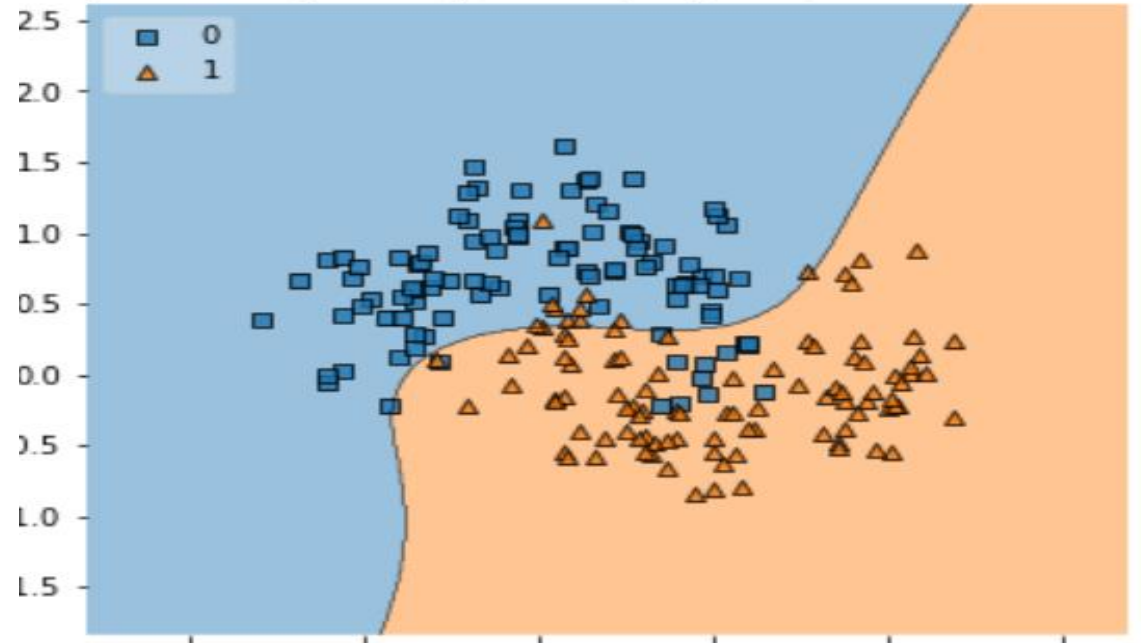
```
from sklearn.preprocessing import PolynomialFeatures
```

Feature Transformation and Boundary Shape

Linear Logistic Regression Decision Boundary



Polynomial Logistic Regression (Degree 3) Decision Boundary



Demo: Visualising Decision Boundaries of Classifiers

Demo: Decision Boundary Visualisation

See How Classifiers Split Data

Objective:

- See how classifiers divide feature space
- Understand effects of assumptions (e.g. linearity, distance)
- Explore underfitting, overfitting and flexibility

Lab Activity:

- Open: `Decision_Boundaries.ipynb`
- Watch the demo, then try different classifiers and settings

Demo: KNN Implementation and Model Evaluation

Demo: KNN Classifier in Action

Understand KNN with Real Data

Objective:

- Apply KNN to a real dataset
- Standardise features
- Find best k using train/test accuracy
- Evaluate with a classification report

Lab Instructions:

- Open: `KNN Classifier.ipynb`
- Follow the implementation steps
- Try different k values
- Review results on train and test sets

Q&A Session



Open floor for questions and clarifications.